# Fawtir mobile: API reference, sign-up to issued invoice

Oct 1, 2026 · @Rixon Agency

Every API the web app calls from sign-up to an issued invoice, as the web calls it today on `main`. Use it with the earlier guide "Fawtir mobile: onboarding to invoice creation", which holds the field rules and screens; this doc holds the calls.

## Conventions

**Base URL.** The web calls `/api` + path on its own origin, for example `/api/v1/auth/login`; behind it sits the test backend `https://api.fawtir.om`. All paths below start at `/v1`. Confirm the production host for mobile with the backend.

| Rule | Detail |
|---|---|
| Headers | `Accept: application/json, application/problem+json`; `Content-Type: application/json` (multipart for file uploads). No language or tenant header. |
| CSRF (web only) | The web echoes cookie `FAWTIR_CSRF` in header `X-CSRF-Token` on every non-GET call made with the session. A missing token gives 403 `csrf-rejected`. Mobile needs this only if it uses cookies (see Open questions). |
| Idempotency | Header `Idempotency-Key: <prefix>-<uuid>`, for example `draft-…`, `payment-…`, `customer-…`. Make one key per form; reuse it after a timeout, network error or 5xx; make a new one only after a 4xx. |
| Versions | Edits send `expectedVersion` (number) in the body. A stale version gives 409. Exception: discarding a draft sends `?expected_version=` in the query. |

| Rule | Detail |
|---|---|
| Errors | RFC 7807 problem JSON. `type` is a URI: read its last segment ( `validation-failed` , `resource-conflict` , `rate-limited` , `authentication-failed` ). The module code is in `code` (invoices) or `error` (some other modules). Field errors: `errors[]` of `{field, message, code}` . Never show `detail` : it is English only. 429 carries `Retry-After` in seconds. |
| Pages | 0-based `page` and `size` . Invoices and customers answer `{content, page, size, totalElements, totalPages}` ; products `{content, page, size, totalElements}` . Leave out empty query values rather than sending `q=` . |
| Timeout | The web gives up after 30 s and shows offline or timeout. |
| Money | Three decimals, half up, on everything shown and sent. Currency is always `OMR` ; show the rial sign, never "OMR". |
| Session expiry | There is no refresh endpoint. Any 401 on a signed-in call means: clear state and sign in again. |

**Open questions for the backend**

1. How mobile signs in: the web sends `portalCookie: true` and gets HttpOnly cookies. Mobile needs a token mode: the login and MFA verify body, where the access token comes back, how long it lives, and how it is sent (likely `Authorization: Bearer` ).
2. Whether CSRF applies to token calls (it should not).
3. The production base URL for mobile.

# 1. Sign-up and onboarding

Ten public calls, no session and no cookies. After email verification every step carries the `onboardingToken` in the JSON body, never in a header. The backend refuses unknown fields, so send exactly the fields listed.

Order: sign up, verify, then status, business profile, (business category and type), business identity, address and contact, final submission. On every app start during onboarding, call status and resume at its `stage`.

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 1.1 | `POST /v1/auth/signup` | `fullName`, `username`, `email`, `phone`, `password`, `languageCode` (`en` or `ar`), `termsAccepted`, `privacyAccepted` (booleans) | 201 `{tenantCode, username, email, languageCode, emailVerificationRequired, emailVerificationExpiresAt, resendAvailableAt}` | 409 `resource-conflict`: username taken. A registered email still answers 201 on purpose. `validation-failed` with `errors[]` per field |
| 1.2 | `POST /v1/auth/signup/verify` | `{tenantCode, email, code}`, code = 6 digits | `{onboardingToken, onboardingTokenExpiresAt}` | 401: wrong or expired code, too many tries (one answer for all) |
| 1.3 | `POST /v1/auth/signup/resend` | `{tenantCode, email}` | 202, no body | Allow resend after 60 s; a code lives 10 min |
| 1.4 | `POST /v1/auth/signup/status` | `{onboardingToken}` | `{stage, businessProfileComplete, businessCategoryComplete, businessTypeComplete, businessIdentityComplete, addressContactComplete, submitted, verificationStatus, loginEnabled}` | 401: token dead |
| 1.5 | `POST /v1/auth/signup/business-profile` | `{onboardingToken, businessSize, invoicingMethod}` | 2xx | as below |
| 1.6 | `POST /v1/auth/signup/business-category` | `{onboardingToken, businessCategory}` | 2xx | as below |
| 1.7 | `POST /v1/auth/signup/business-type` | `{onboardingToken, businessType}` | 2xx | as below |
| 1.8 | `POST /v1/auth/signup/business-identity` | `{onboardingToken, legalCompanyName, commercialRegistrationNumber, taxNumber, invoiceAccent, industrialClassificationCode?}` | 2xx | 409: "A business with this VATIN or CR number is already registered." |

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 1.9 | `POST /v1/auth/signup/address-contact` | `{onboardingToken, addressLine, postalCode?, city, governorate, contactPersonName, contactRole?, contactPhone, contactEmail}` | 2xx | as below |
| 1.10 | `POST /v1/auth/signup/final-submission` | `{onboardingToken}` | 204: received, not yet approved | 409: a step missing, already submitted, or VATIN/CR taken |

### Values and rules

- `fullName` 2 to 200, at least one letter, no digits. `username` matches `^[A-Za-z0-9][A-Za-z0-9._-]{2,63}$`. `email` at most 255.
- `phone` and `contactPhone` are sent as `+968` + 8 digits. Landlines start 22 to 26, mobiles 71 to 79 or 90 to 99.
- `password` 12 to 128 characters with an uppercase letter, a lowercase letter, a number and a symbol. Sent as typed, never trimmed. (The owner has asked the backend to lower the minimum to 8; keep 12 until they confirm.)
- `stage`: `BUSINESS_PROFILE`, `BUSINESS_CATEGORY`, `BUSINESS_TYPE`, `BUSINESS_IDENTITY`, `ADDRESS_CONTACT`, `READY_FOR_SUBMISSION`, `UNDER_REVIEW`, `ACTIVATED`, `REJECTED`. A rejection that reopens the registration reads `READY_FOR_SUBMISSION` again.
- `businessSize`: `JUST_ME`, `PEOPLE_2_TO_10`, `PEOPLE_11_TO_50`, `PEOPLE_50_PLUS`. `invoicingMethod`: `PAPER_OR_MANUAL`, `SPREADSHEETS`, `ACCOUNTING_SOFTWARE`, `FULL_ERP_SYSTEM`.
- `businessCategory`: only `FOOD_AND_BEVERAGE`. `businessType`: `COFFEE_SHOP`, `RESTAURANT`, `BAKERY`, `FAST_FOOD`, `JUICE_BAR`, `DESSERT_SHOP`, `CATERING`.
- POS = no: still post `FOOD_AND_BEVERAGE` and `RESTAURANT`, then go to business identity. The backend requires both and has no POS field yet.
- `legalCompanyName` at most 255. `commercialRegistrationNumber` at most 20, upper-cased. `taxNumber` upper-cased, `^OM(11|12)\d{8}$`.
- `invoiceAccent`: `BLUE`, `GREEN`, `ORANGE`, `PURPLE`, `RED`, `NAVY`. `industrialClassificationCode`: a 6-digit ISIC code; leave it out when empty.

- `addressLine` at most 140, `postalCode` 3 digits, `city` at most 100, `contactPersonName` at most 200, `contactRole` at most 100. Leave out blank optional fields.
- `governorate` is sent as the English name: Muscat, Dhofar, Musandam, Al Buraimi, Ad Dakhiliyah, North Al Batinah, South Al Batinah, Ash Sharqiyah South, Ash Sharqiyah North, Adh Dhahirah, Al Wusta.

**Errors on steps 1.5 to 1.10.** 401: the token is dead; say the sign-up session ended and to contact support (no endpoint issues a new token; it lives about 2 hours). 409: call status again; route to the missing step, or treat it as success when already submitted. `validation-failed`: map `errors[].field` to the form.

## 2. Sign-in and session

Sign-in works once the company is activated. Every sign-in needs a second step, an email code by default. The bodies below are the web's; the token mode for mobile is open question 1.

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 2.1 | `POST /v1/auth/login` | `{username, password, portalCookie: true}` (web). Username trimmed, max 255; password as typed, max 128. No tenant code | `{mfaRequired, mfaChallengeToken?, mfaChallengeExpiresIn? (seconds), mfaState?, mfaRequiredFactor?}`. `mfaState`: `VERIFICATION_REQUIRED` or `ENROLLMENT_REQUIRED`. Factor `EMAIL_OTP` unless the server says `TOTP` | 401: one answer for wrong password, unknown user, locked or disabled account, or company still under review |
| 2.2 | `POST /v1/auth/mfa/otp/request` | `{challengeToken, factor: "EMAIL_OTP"}` | 202, no body. Each request replaces the last code | Allow a resend after 60 s |
| 2.3 | `POST /v1/auth/mfa/verify` | `{challengeToken, factor, code (6 digits), portalCookie: true}` (web) | Session started | 401: the challenge is used up after every attempt, right or wrong; send the user back to 2.1 |
| 2.4 | `POST /v1/auth/mfa/totp/enroll` | `{challengeToken}` | `{secret, otpauthUri, enrollmentChallenge}` | Only when `mfaState` is `ENROLLMENT_REQUIRED` |
| 2.5 | `POST /v1/auth/mfa/totp/confirm` | `{challengeToken, code, enrollmentChallenge}` | `{enabled, backupCodes?}`. Does not sign in: call 2.3 next with factor `TOTP` | |

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 2.6 | `GET /v1/me/entitlements` | none | `{userId, username, tenantId, tenantName?, roles[{id, code, name, systemRole}], permissions[], scopes[], featureFlags{}}` | 401: sign in again |
| 2.7 | `POST /v1/auth/forgot-password` | `{username}`, 3 to 255 | Always 202 | Server limit: 5 per minute per IP |
| 2.8 | `POST /v1/auth/reset-password` | `{token, newPassword}`. The token arrives in the email link after `#token=`; the link lasts 30 min | 204; other sessions are signed out | 401: link invalid, expired or used. 400: password breaks the policy or was used before |
| 2.9 | `POST /v1/auth/logout` | none | 204 | 401: already signed out, treat as success |

**Permissions.** Read `permissions` from 2.6 once after sign-in and hide every action the user may not take (hidden, not disabled). The ones this flow uses: `invoice.draft`, `invoice.issue`, `invoice.payment.manage`, `invoice.attachment.manage`, `invoice.internal-note.read`, `customer.read`, `customer.manage`, `product.manage`, `admin.settings.manage`.

## 3. Company settings

Four resources under `/v1/company-profile`. The web saves them in this order, each only if its fields changed, and stops at the first failure: profile, industrial classification, branding, invoice defaults. Logo upload and removal happen at once, not on Save.

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 3.1 | `GET /v1/company-profile` | `{tenantId, tenantCode, tenantName, legalName, legalNameAr?, tradingName?, taxNumber, commercialRegistrationNumber, country, registeredAddress, registeredAddressLine2?, registeredAddressLine3?, registeredCity?, registeredPostalCode?, registeredCountrySubdivision?, registeredPoBox?, contactPersonName?, contactPhone?, contactEmail?, profileRevision, verificationStatus}` | `verificationStatus`: `PENDING_VERIFICATION`, `VERIFIED`, `REJECTED`. `country` is ISO alpha-2 (`OM`) |
| 3.2 | `PUT /v1/company-profile` | Every field from 3.1 except `tenantId`, `tenantCode`, `profileRevision`, `verificationStatus`, plus `expectedRevision` = `profileRevision` | Replaces the whole profile: send unchanged fields back as they are, empty optionals as `null`. Offer it only while `PENDING_VERIFICATION` or `REJECTED`; refused once `VERIFIED`. The VAT number is never editable |
| 3.3 | `GET` and `PUT /v1/company-profile/industrial-classification` | `{code}`: one 6-digit ISIC code, or `null` | PUT needs `admin.settings.manage`; codes not on the official list are refused. Always editable |
| 3.4 | `GET` and `PUT /v1/company-profile/branding` | GET: `{logo: {id, contentType, byteSize, sha256, widthPx, heightPx} or null, brandColour, receiptMonochrome}`. PUT: `{brandColour, receiptMonochrome}` | Needs `admin.settings.manage`; a 403 on GET means "no logo". Always editable, no re-verification |
| 3.5 | `POST /v1/company-profile/branding/logo` | Multipart, field `file` | The web accepts PNG and SVG up to 2 MB |
| 3.6 | `GET /v1/company-profile/branding/logo` | The image bytes | Add `?v=<logo id>` to refresh a cached copy |
| 3.7 | `DELETE /v1/company-profile/branding/logo` | none | Removes the logo |

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 3.8 | `GET` and `PUT /v1/company-profile/invoice-defaults` | `{language, currency, paymentTerms, footerNote}` | `language`: `AR`, `EN` or `AR_EN`. `currency`: `OMR` only. `paymentTerms`: a code from reference data. Empty values as `null`. No re-verification |

The business activity (3.3) is never asked on an invoice: the backend copies it onto every line (IBR-081-OM), and B2B and B2G invoices are refused without it (`INDUSTRIAL_CLASSIFICATION_REQUIRED`).

## 4. Reference data and lookups

Load reference data and branches once after sign-in. The answers carry codes only; labels come from the app's own English and Arabic text.

| # | Call | Answer | Rules |
|---|---|---|---|
| 4.1 | `GET /v1/reference-data` | `taxCategories[{code, en16931Category, rate, exemptionReasonRequired, active}]`, `units[{code, active}]`, `paymentTerms[{code, netDays, active}]`, `paymentMethods[{code, active}]` | Never offer inactive entries. Keep only VAT categories S, Z, E and O (drop AE and G: not Oman categories). `rate` is a string, for display only: the server sets the rate from the category. `netDays` gives the due date |
| 4.2 | `GET /v1/branches` | A plain array of `{id, code, name, type, city}` | Has no head-office flag. Default the branch to the one with `type` `HEAD_OFFICE` |

| # | Call | Answer | Rules |
|---|---|---|---|
| 4.3 | `GET /v1/code-lists/hs?q=&limit=20` | `{items[{code, name, nameAr?}], total}` | Server search over 14,313 HS codes of 12 digits. Debounce typing (the web waits 250 ms). Always add Other = `000000000000` at the end |

If 4.1 has not answered yet, the web falls back to: S5 (5%), Z0, E0, O0; terms NET0, NET30, NET60; payment methods 30 Bank transfer, 10 Cash, 20 Cheque, 48 Card, 68 Online.

**Lists bundled in the app, not served by the API.** Copy these files from the web repo:

- `src/lib/codeLists/isic.ts`: 1,819 ISIC business activities, English and Arabic; `000000` = Other. There is no ISIC search endpoint yet, so the app searches its own copy.
- `src/lib/codeLists/isicBySubType.ts`: the suggested ISIC code per sign-up sub type.
- `src/lib/codeLists/serviceTypes.ts`: 21 service types (CL-12-OM); `00000000` = Other.
- `src/lib/codeLists/taxReasons.ts`: VATZR-OM-01 to 16 for zero-rated, VATEX-OM-01 to 12 for exempt.

# 5. Customers and products

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 5.1 | `GET /v1/customers?status=ACTIVE&type=&search=&sort=&page=&size=` | Page of `{id, type, displayName, peppolDelivery, invoiceEligible?, missingInvoiceFields?, status, updatedAt, profile{details, contact, deliveryPreferences, invoiceAddress}}` | Needs `customer.read`. The search parameter is `search`, not `q`. `type`: `BUSINESS`, `GOVERNMENT`, `CONSUMER`. `sort`: `RECENT`, `NAME_ASC`, `NAME_DESC`. `size` must be 10, 25, 50 or 100, else 400 |
| 5.2 | `GET /v1/customers/{id}` | The row plus the full `profile` and `version` | `type` decides the invoice: `CONSUMER` gives a simplified invoice |

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 5.3 | `POST /v1/customers` | `{type, logo, details, contact, billingDefaults, deliveryPreferences, invoiceAddress, sameAsInvoicingAddress, deliveryAddress, customerVisibleNote}`, flat, no `profile` wrapper | Needs `customer.manage` and `Idempotency-Key: customer-<uuid>`. A consumer (walk-in) is `type: CONSUMER` with `details.firstName` and `details.lastName`, plus an email or phone |
| 5.4 | `PUT /v1/customers/{id}` | Same body plus `expectedVersion` | Needs `customer.manage`. 409 on a stale version |
| 5.5 | `GET /v1/products?q=&size=20` | Page of `{id, code, name, description, defaultUnitCode, defaultUnitPrice, defaultTaxCategory, defaultTaxRate, active, version}` | Active products whose code or name contains `q`. Price excludes VAT |
| 5.6 | `POST /v1/products` | `{code, name, defaultUnitCode, defaultUnitPrice, defaultTaxCategory, defaultTaxRate}` | Needs `product.manage`. Name max 500, code max 100, price at least 0 at 3 decimals. Rate `null` for E and O. `PRODUCT_CODE_DUPLICATE`: code taken |

**Customer body details** (5.3):

- `details`: `firstName`, `lastName` (consumer); `legalRegisteredName` (business); `entityMinistryName`, `entityMinistryCode`, `defaultBuyerReference`, `defaultPurchaseOrderNumber`, `defaultContractTenderReference` (government); `registeredForVat`, `vatRegistrationNumber` (upper-cased, only when registered), `registeredOnPeppol`, `peppolParticipantId {scheme, value}`, `specialZoneLicenceNumber`, `industrialClassificationCode`. For a consumer, `registeredForVat` and `registeredOnPeppol` are `null`.
- `contact`: `contactPerson`, `roleTitle`, `emailRecipients[]` (max 10), `telephone`, `mobilePhone`, `preferredLanguage` (`en`, `ar` or `null`).
- `billingDefaults`: `defaultCurrency`, `defaultTaxCategory`, `paymentTerms`, `defaultPaymentMethod`.
- `deliveryPreferences`: `sendingMethod` (`EMAIL_PDF`, `PEPPOL`, `QR_RECEIPT`; never `PEPPOL` for a consumer), `emailPdfCopy`, `automaticPaymentReminders`.
- Addresses: `attention`, `countryCode`, `cityName`, `countrySubentity`, `streetName`, `buildingNumber`, `additionalStreetName`, `poBox`, `postalCode`.
- Peppol ID scheme 0248 or 0242; with 0248 the value is the VAT number `OM11` + 8 digits. Show it as `0248:` + value, never `0248:` twice.

**Errors.** `errors[].field` gives a path such as `details.vatRegistrationNumber`, with `code` `REQUIRED`, `TOO_LONG`, `INVALID_FORMAT`, `INVALID_CHARACTERS`, `DUPLICATE`, `TOO_MANY`. A 409 on create is a duplicate VAT number or Peppol ID.

**Picking a product on a line** fills `productId`, `productCode`, description (the name), unit, the VAT category matching category and rate, and the price (VAT added if the line is entered VAT-inclusive). Issuing refuses a line whose product code no longer matches the product.

## 6. Invoice: draft to issued

An invoice is a draft first; issuing numbers it and fixes it. A draft stores no figures: the totals come from preview. Needs `invoice.draft`, and `invoice.issue` to issue.

### Flow the web follows

1. Create the draft (6.1), or update it (6.2) when it exists.
2. Upload queued attachments one by one, now that the draft has an id (7.6).
3. Preview (6.5) for the totals of record.
4. Issue (6.6) with the dates.
5. Optional, after issue: record the payment (7.3) and mark it sent (7.5). If either fails, the invoice stays issued.

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 6.1 | `POST /v1/invoice-drafts` | Header `Idempotency-Key: draft-<uuid>`. Body `{type: "INVOICE", draft: <draft body>}` | The draft: `{id, version, authoringState, customerId, branchId, currency, invoiceNumber (null), type, headerDiscount, details, lines[]}` | `errors[].field` such as `draft.lines[1].unitPrice`, mapped to the line |
| 6.2 | `PUT /v1/invoice-drafts/{id}` | `{expectedVersion, draft: <draft body>}` | The draft; keep its new `version` | `STALE_INVOICE_VERSION`: someone else saved it; reload, no merge |
| 6.3 | `GET /v1/invoice-drafts/{id}` | none | The draft. Answer lines are flat: `discountType`, `discountValue`, `taxCategoryCode`, `en16931Category`, `taxRate`, `exemptionReason`, `exemptionReasonCode` | |
| 6.4 | `DELETE /v1/invoice-drafts/{id}?expected_version=<n>` | none | Draft set to discarded | `STALE_INVOICE_VERSION` |

| # | Call | Body | Answer | Errors to handle |
|---|---|---|---|---|
| 6.5 | `POST /v1/invoice-drafts/{id}/preview` | none; prices the last saved version | `{currency, lines[{lineNumber, grossAmount, lineDiscount, allocatedHeaderDiscount, taxableAmount, vatAmount, lineNetAmount}], totals{subtotal, itemLevelDiscount, headerDiscount, totalDiscount, taxableAmount, vatAmount, totalAmount}}` | See the preview table below |
| 6.6 | `POST /v1/invoice-drafts/{id}/issue` | Header `Idempotency-Key: draft-<uuid>`. Body `{expectedVersion, issueDate, dueDate}` as `YYYY-MM-DD`, nothing else | The issued invoice; use `id` and `invoiceNumber` | See section 9 |

**Draft body** (unknown fields are refused):

```
{
  "customerId": "…", "branchId": "…", "currency": "OMR", "vatMode":
"EXCLUSIVE",
  "headerDiscount": { "type": "PERCENTAGE", "value": 0 },
  "details": {
    "transactionType": "FULL_TAX",
    "paymentTerms": { "code": "NET30", "label": "Net 30 days" },
    "paymentMethod": { "code": "30", "label": "Bank transfer" },
    "deliveryDate": null, "subject": null, "purchaseOrderReference": null,
    "customerVisibleNote": null, "internalNote": ""
  },
  "lines": [{
    "lineNumber": 1, "productId": "…", "productCode": "BEANS-1KG",
    "description": "Arabic coffee beans 1 kg", "unitCode": "KGM",
    "quantity": 2, "unitPrice": 6.5,
    "discount": { "type": "PERCENTAGE", "value": 0 },
    "taxCategory": { "code": "S5", "en16931Category": "S", "rate": 5,
                     "exemptionReason": null, "exemptionReasonCode": null },
    "classification": { "goodsOrService": "G", "hsCode": "090121000000" }
  }]
}
```

### Draft body rules

- `currency` is always `OMR`; `vatMode` is always `EXCLUSIVE` (the server refuses `INCLUSIVE`). Prices are always sent excluding VAT: convert an incl. price with price × 100 ÷ (100 + rate).
- `transactionType`: `SIMPLIFIED` for a consumer customer, else `FULL_TAX`.
- `paymentTerms` and `paymentMethod` are `{code, label}` or `null`; the label is the text the user saw. Payment methods: 30 Bank transfer (default), 10 Cash, 20 Cheque, 48 Card, 68 Online.
- `internalNote`: leave it out to keep the stored note, send `""` to clear it. It is read back only through 7.7.
- `productId` and `productCode` are both sent or both left out (free-text line).
- `discount` and `headerDiscount`: `type` `PERCENTAGE` or `FIXED_AMOUNT`.
- VAT: S sends no reason. Z sends `exemptionReasonCode` VATZR-OM-xx; E sends VATEX-OM-xx (with `exemptionReason: null`). O sends free text in `exemptionReason` (optional, max 500) with the code `null`. Lines at one category and rate must carry the same reason.
- `classification` (business and government only): goods `{goodsOrService: "G", hsCode}` with 12 digits, required on every goods line before issue (IBR-079-OM); services `{goodsOrService: "S"}`, plus `serviceTypeCode` (8 digits) only for an export of services. Consumer service lines send no classification. Never send `industrialClassificationCode`: the backend copies the company's.
- Before save: customer and branch set, every line has a description, quantity above 0, price 0 or more, quantity × price at most 999,999,999,999.999.

**Issue rules.** `issueDate` not after today in Muscat time and in the current year. `dueDate` is worked out on the device from the terms (`netDays`) and is not before `issueDate`. Issuing does not send the invoice to the customer.

**Preview refusals**

| Code | Meaning |
|---|---|
| `INVOICE_LINES_REQUIRED` | Add at least one line |
| `LINE_DISCOUNT_EXCEEDS_BASE` | A line discount is more than the line |
| `HEADER_DISCOUNT_EXCEEDS_BASE` | The document discount is more than the total |
| `LINE_DISCOUNT_PERCENTAGE_INVALID` | A line discount is over 100% |

| Code | Meaning |
|---|---|
| `HEADER_DISCOUNT_PERCENTAGE_INVALID` | The document discount is over 100% |
| `EXEMPTION_REASON_REQUIRED` | A zero-rated or exempt line needs its reason |
| `TAX_CATEGORY_METADATA_CONFLICT` | Two lines at one rate have different categories |
| `TAX_CATEGORY_REASON_CONFLICT` | Two lines at one category and rate have different reasons |

**Totals on the device** (live estimate only; the preview is the figure of record): line gross = quantity × price; line discount capped between 0 and gross; document discount next (a percentage per line, a fixed amount shared by each line's value); VAT per rate on the grouped taxable amount, then shared back to the lines; half up at 3 decimals, more precision inside. The exact math is in `src/lib/invoiceMath.ts`.

**Duplicate.** To copy an issued invoice, read it with 6.3 and start a new unsaved form from it, without dates, number or payments. Saving creates a new draft.

## 7. After issue: reading, paying, files

All under `/v1/authored-invoices/{id}`.

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 7.1 | `GET /v1/authored-invoices/{id}` | `{id, type, version, authoringState, deliveryState, customerId, branchId, currency, vatMode, invoiceNumber, issueDate, dueDate, issuedAt, details, headerDiscount, lines[], totals}` | `totals` is null until issued. `deliveryState` is `NOT_SUBMITTED` or `SENT_MANUALLY` today |
| 7.2 | `GET …/receivable` | `{documentTotal, paidAmount, creditedAmount, outstandingAmount, currency, paymentState, overdue, dueDate}` | `paymentState`: `UNPAID`, `PARTIALLY_PAID`, `PAID`, `NOT_APPLICABLE` |

| # | Call | Body or answer | Rules |
|---|---|---|---|
| 7.3 | `POST …/payments` | Header `Idempotency-Key: payment-<uuid>` (16 to 128 of `A-Z a-z 0-9 . _ : -`). Body `{amount, methodCode, paymentReference, paymentDate}` | Needs `invoice.payment.manage`. Issued and not fully paid only. `amount` at most 3 decimals; `methodCode` default `30`; `paymentReference` or `null`; `paymentDate` `YYYY-MM-DD` |
| 7.4 | `GET …/payments` and `POST …/payments/{paymentId}/reverse` | List: `[{id, entryType, reversalOfPaymentId, amount, currency, methodCode, paymentReference, paymentDate, occurredAt}]`. Reverse: `Idempotency-Key`, no body | `entryType`: `PAYMENT` or `REVERSAL`. Not asked for drafts |
| 7.5 | `POST …/manual-delivery` | No body. Answer `{recordedAt}` | Marks the invoice sent by hand. Marking twice returns the first record |
| 7.6 | `POST …/attachments` (multipart field `file`), `GET …/attachments`, `GET` and `DELETE …/attachments/{attachmentId}` | Upload answer `{id, filename, contentType, byteSize, sha256, attachedAt, attachedByName?}` | Needs `invoice.attachment.manage`. Uploads only while a draft (`INVOICE_ATTACHMENT_REQUIRES_DRAFT`). The web accepts `.pdf .jpg .jpeg .xlsx`, max 10 MB. Delete refused under legal hold (`INVOICE_ATTACHMENT_NOT_REMOVABLE`) |
| 7.7 | `GET …/internal-note` | `{internalNote}` | Needs `invoice.internal-note.read`; every read is audited, so read it only when shown |
| 7.8 | `GET …/history` | `{entries[{id, eventType, outcome, invoiceVersion, actorId, occurredAt}], truncated, limit}` | For an activity list |
| 7.9 | `GET /v1/authored-invoices?type=INVOICE&state=&payment=&q=&sort=&page=&size=` | Page of rows with `invoiceNumber`, `customerName`, `issueDate`, `dueDate`, `totals.totalAmount`, `payment{state, overdue, daysOverdue, outstandingAmount, …}` | `state`: comma list of `DRAFT`, `ISSUED`, `CANCELLED`. `payment`: `AWAITING`, `OVERDUE`, `PAID`. `sort`: `UPDATED`, `ISSUED_NEWEST`, `ISSUED_OLDEST`, `DUE_SOONEST`, `AMOUNT_HIGHEST`, `AMOUNT_LOWEST`, `CUSTOMER` |
| 7.10 | `GET /v1/authored-invoices/summary?type=INVOICE` | Counts and totals: `all`, `drafts`, `awaiting`, `overdue`, `paid`, `outstandingTotal`, `overdueTotal`, `paidTotal`, … | For list tabs and cards |

### Payment refusals (7.3)

| Code | Meaning |
|---|---|
| `PAYMENT_EXCEEDS_OUTSTANDING` | More than is still owed |
| `PAYMENT_AMOUNT_SCALE_INVALID` | More than 3 decimals |
| `PAYMENT_DATE_IN_FUTURE` | Payment date in the future |
| `PAYMENT_DATE_BEFORE_ISSUE_DATE` | Payment date before the invoice date |
| `PAYMENT_METHOD_INVALID` | Unknown method code |
| `PERIOD_CLOSED` | The date is in a closed period |
| `PAYMENT_REQUIRES_ISSUED_INVOICE` | The invoice is not issued |
| `PAYMENT_ALREADY_REVERSED`, `PAYMENT_NOT_FOUND` | On reverse |

There is no PDF endpoint yet, and no Peppol send endpoint: issuing does not deliver the invoice.

## 8. Bulk invoice import

An Excel file creates drafts only, never issued invoices, and is taken whole or not at all. Needs `invoice.draft`; `CONSUMER` also needs `customer.manage`. A 404 means import is not live on that server yet. Limits: `.xlsx`, 10 MB, 2,000 rows; over 50 rows it runs in the background.

| # | Call | Body or answer |
|---|---|---|
| 8.1 | `GET /v1/invoices/import/template?kind=&language=` | The `.xlsx` template. `kind`: `BUSINESS`, `CONSUMER`, `GOVERNMENT`; `language`: `en` or `ar` |
| 8.2 | `POST /v1/invoices/import/check` | Multipart: `file`, `kind`, `language`. Answers a batch |
| 8.3 | `GET /v1/invoices/import/{batchId}` | The batch; poll about every 1.5 s while `state` is `CHECKING` or `CREATING` |

| # | Call | Body or answer |
|---|---|---|
| 8.4 | `POST /v1/invoices/import/{batchId}/commit` | `{classifications: [{description, itemType, code}]}`; `itemType` `GOODS` or `SERVICE`; `code` a 12-digit HS code (`000000000000` = Other) or an 8-digit service type |
| 8.5 | `GET /v1/invoices/import/{batchId}/report` | The report file |
| 8.6 | `DELETE /v1/invoices/import/{batchId}` | Cancels the batch |

**Batch:** `{id, kind, state, filename, fileSha256, duplicateOf{batchId, uploadedAt, uploadedBy} or null, totalRows, processedRows, errors[{rowNumber, column, message}], summary{invoices, lines, total, byCustomer[]}, inferred[{rowNumbers, column, value, matchedTo}], unresolved[{description, itemType, rowCount}], created[{reference, invoiceId, draftNumber, customerName, total}]}`. `state`: `CHECKING`, `CHECKED`, `CREATING`, `CREATED`, `CANCELLED`. Any error blocks the whole file. Ask once for an HS code or service type per `unresolved` description, then commit.

## 9. Issue refusals and checklist

Codes from 6.6, read from `code` in the problem answer. Codes marked with a field go under that date field; the rest are a message on the form. Any other 409 `resource-conflict`: "Your account can't issue invoices right now" with the code.

| Code | Field | What to tell the user |
|---|---|---|
| `ISSUE_DATE_IN_FUTURE` | issue date | An invoice can't be dated in the future |
| `ISSUE_DATE_OUTSIDE_CURRENT_YEAR` | issue date | The issue date must be in this calendar year |
| `PERIOD_CLOSED` | issue date | That date is in a closed accounting period |

| Code | Field | What to tell the user |
|---|---|---|
| `DUE_DATE_BEFORE_ISSUE_DATE` | due date | The due date can't be before the issue date |
| `COMPANY_VERIFICATION_PENDING`, `SELLER_NOT_VERIFIED` | | The company is still being verified |
| `COMPANY_VERIFICATION_REJECTED` | | Verification was declined; contact support |
| `TENANT_NOT_ACTIVE`, `TENANT_DISABLED` | | The account isn't active |
| `QUOTA_EXCEEDED` | | This month's invoice allowance is used up |
| `TENANT_TIMEZONE_UNAVAILABLE` | | The company time zone isn't set |
| `SELLER_PARTICIPANT_NOT_VERIFIED` | | The Peppol registration isn't verified yet |
| `INDUSTRIAL_CLASSIFICATION_REQUIRED` | | Set the business activity in Company settings |
| `CUSTOMER_NOT_INVOICE_ELIGIBLE` | | The customer is missing details |
| `CUSTOMER_NOT_SELECTABLE` | | This customer can't be invoiced |
| `PAYMENT_TERMS_REQUIRED` | | Choose payment terms before issuing |
| `BRANCH_NOT_ACTIVE` | | The branch is no longer active |
| `PRODUCT_NOT_FOUND`, `PRODUCT_CODE_MISMATCH` | | A product no longer matches the catalogue |
| `PRODUCT_ARCHIVED` | | A product was archived |
| `STALE_INVOICE_VERSION` | | Someone changed this draft; reload |
| `INVOICE_NOT_ISSUABLE` | | It is no longer a draft |
| `REFERENCE_CATALOGUE_NOT_AVAILABLE` | | Issuing isn't available on this server |

The web's English and Arabic wording for all of these is in `src/i18n/locales/en.json` and `ar.json`; the code-to-message map is in `src/lib/invoiceIssue.ts`.

### Checklist

- [ ] Sign-up: ten calls, token in the body, resume from `stage`, 409 re-reads status
- [ ] Phone sent as `+968` + 8 digits; landlines 22 to 26, mobiles 71 to 79 or 90 to 99
- [ ] Sign-in with the email code; token mode agreed with the backend
- [ ] Permissions read once from entitlements; actions the user can't take are hidden
- [ ] Company settings: four resources, saved in order; ISIC always editable
- [ ] Reference data filtered to S, Z, E, O and to active entries
- [ ] ISIC, service types and VAT reasons bundled in the app
- [ ] Draft create and update with idempotency key and `expectedVersion`
- [ ] Attachments uploaded after the draft is saved
- [ ] Totals from preview, three decimals, rial sign
- [ ] Issue with dates in Muscat time; every refusal code mapped
- [ ] Optional mark paid and mark sent after issue
- [ ] English and Arabic for every label